

Distributed In-Memory Cache with Strong Consistency Guarantees

(Distributed In-Memory Cache
with Strong Consistency Guarantees)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

20 grudnia 2025

Abstract

When designing distributed systems we need to make trade offs. The basics of such trade offs are defined in CAP theorem, where we cannot guarantee three of Consistency, Network partition tolerance and Availability. Most of implementations try to ensure Availability and Network partition tolerance, for maximizing throughput and speed of cache. In this paper, I will focus on Network Partition tolerance and Consistency. I want to achieve Strong Consistency.

My architecture splits the system into layers. First layer is a Control Plane, responsible for managing the cluster and for the consensus. Raft algorithm is used for consensus between the servers in the cluster. The next layer is the Data Plane. It is responsible for splitting data into shards. Each of these shards is called a data node. Another part of the system is Smart Client. It is locally storing the cluster topology, fetched from Control Plane to manually calculate to which data node should one send a request. Thanks to that, our system minimizes network overhead and latency. The system is implemented in Go language with gRPC for internode communication and http for client-server communication.

Implementation has been tested with multiple tests. The Raft algorithm was a very detailed and throughoutly tested, as this is the base of the entire system. Then, there were written End-To-End tests, alongside Failure injection, such as network partitions, single servers and nodes failures, stops and restarts. There were also benchmarks, comparing the implementation to known cache systems, such as Redis. Results show that systems meets the conditions of Strong consistency and Network Partition and also offers competitive throughput compared to industry-standard solutions.

Gdy tworzy się systemy rozproszone trzeba wziąć pod uwagę różne kompromisy które trzeba poczynić przy tworzeniu takiego systemu. Podstawowymi kompromisami w takich systemach jest wybór między Spójnością, Dostępnością oraz Tolerancją na podział sieci.

CAP theorem mówi nam, że system nie jest w stanie spełnić jednocześnie wszystkich trzech cech, więc musimy zrezygnować z jednej z nich. Ja w swojej pracy postawiłem na Spójność oraz Tolerancję na podział sieci. Większość rozwiązań produkcyjnych stawia na dostępność zamiast spójności, aby przyspieszyć działanie i ogólny czas dostępu do systemu.

Moja architektura dzieli system na warstwy. Pierwszą z nich jest Control Plane, który jest odpowiedzialny za zarządzanie klasterem oraz konsensus, dzięki algorytmowi Raft, pomiędzy serwerami wchodzącymi w jego skład. Kolejną warstwą jest Data Plane który ma rozdrobione węzły danych. Waznym elementem systemu jest Smart Client, który zapisuje lokalnie topologię klastra, którą pobiera z Control Plane i dzięki temu wysyła zapytania do odpowiednich węzłów danych, minimalizując opóźnienie i ilość zapytań sieciowych.

Cały system jest zaimplementowany w Go oraz używa gRPC do komunikacji między serwerami oraz http do komunikacji między serwerami a klientami.

Implementacja została przetestowana przeróżnymi testami. Bardzo dokładnie został przetestowany algorytm Raft, który jest podstawą całego systemu. Następnie zostały przeprowadzone testy End-to-End, testy na wprowadzanie celowego błędu do systemu, np. podział sieci, restart serwerów. Wykonane zostały również benchmarki, które porównały implementację systemu do popularnych implementacji stosowanych w branży. Wyniki demonstrują, że system spełnia założone wymagania przy wydajnościach porównywalnych do popularnych implementacji.

Contents

1	Introduction	9
1.1	Distributed systems and the problem with data consistency	9
1.2	Objective and scope of the thesis	9
1.3	Technology stack choice	10
1.4	Structure of this thesis	10
2	Theoretical Foundations and Overview of Solutions	13
2.1	Consistency models in distributed systems	13
2.1.1	The CAP theorem and trade-offs	13
2.1.2	Strong Consistency (Linearizability) vs. Eventual Consistency	14
2.2	Split brain problem	15
2.3	Consensus algorithms	15
2.3.1	Paxos vs. Raft – Comparison of understanding and implementation	16
2.4	Detailed analysis of the Raft algorithm	16
2.5	Cache system architectures	17
2.5.1	Sharding and Consistent Hashing	17
2.5.2	Role of the client: Thin Client vs. Smart Client	18
2.6	Overview of existing solutions (Redis Cluster, Etcd) – Analysis in terms of consistency	19
2.6.1	Redis Cluster	19
2.6.2	Etcd	19
2.6.3	Analysis of hashing approaches	20
2.6.4	Analysis of industry inspirations	21

3	Requirements analysis and Software Architectre Design	23
3.1	Functional and non-functional requirements	23
3.2	High level architecture	24
3.2.1	Separation of Control Plane and Data Plane	24
3.2.2	gRPC communication protocol and interface definition	24
3.3	Design of Control Plane module	25
3.3.1	Cluster topology and metadata menagement	25
3.3.2	Node failure detection mechanism	25
3.4	Smart Client design	25
3.4.1	Query routing strategy and topology caching	25
3.4.2	Handling retries and Failover mechanism	26
4	Implementation of the Raft Consensus Module (pkg/raft)	27
4.1	Raft node state machine	27
4.1.1	Implementation of roles: Follower, Candidate, Leader	27
4.1.2	Concurrency and Locking Model	28
4.1.3	Leader election mechanism (election.go)	28
4.2	Log replication logic (replicator.go)	28
4.2.1	Isolated Replicators and The Pulse Protocol	28
4.2.2	gRPC Connection Management	29
4.2.3	Committing entries (Commit Index) and applying to the state machine	29
4.2.4	The Log Request (AppendEntries)	30
4.3	Asynchronous Persistence Management	30
4.3.1	Writing Raft state to disk (persistance.go)	30
4.3.2	State Recovery	30
4.4	Log Compaction and Snapshotting	30
4.4.1	Snapshot Trigger and Creation	31
4.4.2	Chunked Snapshot Transfer (InstallSnapshot)	31
4.5	System Configuration and Environment Parameters	31

5	Implementation of Control Plane and Data Plane	33
5.1	Implementation of Control Plane	33
5.1.1	Topology state machine	33
5.1.2	Node health monitoring	34
5.1.3	Shard rebalancing	34
5.1.4	The NoOp Command	35
5.1.5	Controller HTTP API Reference	35
5.2	Data Plane	36
5.2.1	Concurrent Data Store	36
5.2.2	Lease management and fencing	37
5.2.3	Synchronization and Consistency	37
5.2.4	Data transfer protocol	38
5.2.5	Data Node HTTP API Reference	38
5.2.6	Smart client mechanism	39
5.2.7	Adaptive Retry and Backoff Strategy	40
5.2.8	Client-side idempotency	40
5.3	Control Plane API Layer (pkg/api)	41
5.3.1	Bridging Raft and HTTP	41
5.3.2	Idempotency and Resilience	42
5.3.3	Control Plane Client API Reference	42
6	Correctness verification and Fault injection testing	43
6.1	Methodology for testing distributed systems	43
6.1.1	Docker-based Simulation Environment	43
6.2	Unit and integration tests for the Raft implementation	44
6.3	Simulation of failures and network partitioning	44
6.3.1	Fault Injection and End-to-End Test Suite	45
6.4	Continuous Integration (CI/CD)	46
7	Performance analysis and Comparison with Existing Solutions	47
7.1	Benchmarks	47

7.1.1	Benchmark Toolset	47
7.2	Throughput study (RPS) depending on the number of clients and shards	48
7.3	Analysis of Raft protocol overhead on response time (Latency) . . .	48
7.4	Comparative study	49
7.4.1	Performance comparison with Etcd	49
7.4.2	Performance comparison with Redis	49
7.4.3	Conclusions from the comparative analysis	50
8	Summary	51
8.1	Evaluation of the degree of achievement of the work's objectives . .	51
8.1.1	Encountered implementation problems and challenges	51
8.2	Directions for further development of the project	52
	Bibliography	53
	Test	

Chapter 1

Introduction

1.1 Distributed systems and the problem with data consistency

When we are collecting data from our system the amount might grow infinitely, yet computers have limited amount of RAM and Disk space they can support at once, whether from motherboard limits or just physical limits and law of nature. One day developers might have to come to a decision to start scaling the application. In the beginning, the application might be scaled to just two or three servers, but as more and more data is collected, the entire ecosystem might span tens and hundreds of servers. Some of the data that is collected might be needed often, very often. If we need data often, then we usually put it in cache, as it's fast and we don't want to fetch everything from slow databases. But as the amount of data grows even larger, even the cache cannot keep all the data we need. Then we need to scale cache. And there comes a problem. Assume we have two servers that split the data for cache. One of them have a value 3 for key 1, the other have value 4 for key 1. Which answer is correct? This is the problem of data consistency in the system.

1.2 Objective and scope of the thesis

The goal of thesis is to build a distributed in-memory cache that assures strong consistency of the data - there exist a linearizable moment in the system, where every single server agrees to data they share. As this is cache, and not database, we do not guarantee data durability. If we data is lost, then the user has to refetch data from database and put it in the cache again. We only guarantee, that whenever user asks for the data and receives a response, this response will be correct.

1.3 Technology stack choice

I choose to go with Go for a programming language. This is a language that is based around concurrency which is the base of this thesis. Both single servers are operating concurrently, and they also talk to each other in concurrent way. Go is also very good for network operations, which there are a lot. Other considered choices were C++ and Java. The first was rejected because of it's complexity and the scale of this project aswell as author's skills in using it, the latter because it's slower than Go and less developer friendly.

For the inter-node communication the gRPC was used, because of it's speed. For server-client communication, http was used, because of it's ease of usage.

1.4 Structure of this thesis

Chapter 2 presents theoretical background necessary to understand the problem of state management in distributed systems. It discusses the CAP theorem, data consistency models, consensus algorithms - what are they, why we need them. It will also briefly compare two of the most popular consensus algorithms and describe what the chosen one is doing. It also reviews existing solutions for architectures, sharding, client types and actual systems used in the industry.

Chapter 3 presents more detailed requirements of the project and the system design. It defines high-level architecture, separation of Control Plane from Data plane. It also details communication between servers and the specific design of client.

Chapter 4 presents a detailed implementation of Raft consensus algorithm. It details implementation of basic parts of the system aswell as more advanced.

Chapter 5 covers the implementation details of the remaining system components. It describes the logic of the Control Plane, the Data Nodes and the client library. Special attention is given to mechanisms of shard rebalancing and failure detection.

Chapter 6 is dedidacted to testing and verification of the system. It describes the testing methology, including unit tests, end to end tests, integration tests and failure injection tests used to validate system correctness and stability.

Chapter 7 presents the performance analysis and benchmarks in comparison to other industry-standard systems. It contains the results of benchmark tests measuring latency and throughput (requests per second).

Chapter 8 summarizes the thesis, evaluating the degree to which the project goals were met, discussing the encountered challenges and potential directions for future development.

Chapter 2

Theoretical Foundations and Overview of Solutions

2.1 Consistency models in distributed systems

Distributed systems need to agree on the data they have. Because of how the systems are built, we define three possible consistency models, ranked by their strength:

- Strong consistency - in this model all nodes agree on the order in which operations were done. Reads will always return the most recent version of the data and when update occurs, then this model will make sure that every other server will reflect on that. Because of these guarantees, this model has limitation in terms of how fast it allows the system to work.
- Sequential consistency - all nodes see all operations in the same global order, but that order is not required to be the actual real-time order. Operations happen in sequential, single, unified order.
- Casual consistency - ensures all nodes see the correct order for operations that are casually related, i.e. one operation is influenced by another
- Eventual consistency - The most relaxed model, it prioritizes availability and performance. It allows that data between server might be conflicting, but if no updates occur, eventually these conflicts will resolve.

2.1.1 The CAP theorem and trade-offs

CAP theorem, formulated by Eric Brewer in [1], says that a networked system that shares-data, can strictly provide only two of three properties:

- **Consistency (C):** Reads will receive the most recent write or an error. If a

system is consistent, then all nodes will see the same data at the same time, just like a single-node database would.

- **Availability (A):** Request will always receive a non-error response, without the guarantee that it can contain untrue data. The base of availability is that the system remains operational for reads and updates, even if some nodes fail.
- **Partition Tolerance (P):** The system will work even if a network failure occurs or some nodes crash or there are very big delays.

In real-world distributed systems, we cannot have Consistent and Available system, because we cannot guarantee that network will not partition. If a hardware failure or network failure occurs, then there is inevitable Network Partition. Hence in real-world systems, we will choose between Consistency and Availability. Most cache systems focus on Availability, because we want to maximize speed, and if we want consistency, we should use database. However, there are cases, where we need Consistency for our cache, for example in banking applications.

So the systems are either:

1. **CP (Consistency + Partition Tolerance):** When partition occurs, system will shut down all non-consistent nodes or refuse writes if the system cannot support them any longer. This approach prioritizes safety over throughput or uptime.
2. **AP (Availability + Partition Tolerance):** When a partition occurs, system will continue to accept write and read requests, with probability that it will return untrue data. This approach prioritizes throughput, uptime and user-experience over correctness of the data.

So my system will be a CP system.

2.1.2 Strong Consistency (Linearizability) vs. Eventual Consistency

Strong Consistency (Linearizability)

Linearizability is the strongest guarantee for consistency for single-object requests. It gives a illusion that system is in fact a single copy of data and the data is not distributed. It gives us a guarantee that once write operation is completed sucessfully, all subsequent read operations will return that value, or a newer value if any update happened.

Linearizability (the formal definition of Strong Consistency) says:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

Mathematically, when operation $W(x)$ completes at time t_1 , then for any given operation $R(x)$ starting at time $t_2 > t_1$ must see the effects of $W(x)$.

Eventual Consistency

In eventually consistent systems, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value. This model allows for temporary inconsistencies where different nodes serve different versions of data. This approach enables high availability and low latency because writes can be accepted by a single node without waiting for other nodes to accept and make a consensus. This approach, however, requires to add logic for conflict resolution strategies when merging data from different nodes.

2.2 Split brain problem

Split-brain is a problem in distributed system where a network partition itself into two or more disconnected graph of nodes. This makes the groups unable to communicate with each other and each group may incorrenctly conclude that the other nodes from different groups have failed. This in consequence may start tasks for each groups to find it's leader or primary node. There are multiple ideas to manage this problem. Raft algorithm for example only allows the node to become a leader if more than half of total nodes agrees. The other possibility is to use technique called fencing. This technique is a mechanism where a node physically disables its peer to ensure that it cannot write to shared storage.

2.3 Consensus algorithms

Consensus algorithm is as the name suggest, algorithm that takes care of the state of the data and make sure than all nodes agree on consensus. It is backbone of Consistent systems.

2.3.1 Paxos vs. Raft – Comparison of understanding and implementation

Paxos is an algorithm created by Leslie Lamport and historically the most used algorithm for distributed consensus because of long it has been available. It was mathematically proven to be safe and efficient. However it's also a very complex to understand algorithm and hard to implement.

Raft is more decent algorithm, from 2014, that was created to be easier to implement than Paxos. It offers the same performance as Paxos, but less safety as not all parts of this algorithm have been formally proven [2], however it's still widely used in many modern distributed systems such as CockroachDB[3], MongoDB[4], Neo4J[5], RabbitMQ[6]. It decomposes the problem of consensus into smaller, independent sub-problems of Leader election and Log replication.

Key Differences:

- **Strong Leader:** Raft imposes a stronger leadership model. Log is always send from leaders down to followers. This simplifies the management of the replicated log compared to Paxos, where data can be send in complex peer-to-peer patterns.
- **Log Continuity:** Raft guarantees that if a follower has an entry at index N , it also has all entries from 1 to $N - 1$ identical to the leader. Paxos allows for gaps in the log that must be filled later, increasing implementation complexity.

2.4 Detailed analysis of the Raft algorithm

Raft ensures that a cluster of N servers can tolerate F failures, where $N = 2F + 1$.

Leader Election

Raft uses a heartbeat mechanism to maintain leadership. If no heartbeat is received in some configurable time window, then server can assume the leader is dead and start a new election. The election start by the node incrementing its current **Term** (a logical clock used to detect stale information), changing its current state to the Candidate state, voting for itself, and broadcasting **RequestVote** RPCs to all other nodes. A candidate wins if it receives votes from a majority of the cluster. We want to have randomized timeout for election start because we want to minize the probability of multiple nodes starting election at the same and because of that not one receiving a majority, which would cause system stalls.

Log Replication

Once elected, the leader accepts client commands. A command flows through the following lifecycle:

1. The leader appends the command to its local logs.
2. It sends **AppendEntries** RPCs to all followers.
3. Once a majority of followers acknowledge the entry, the leader marks the entry as **committed**.
4. The leader applies the entry to its state machine and returns the result to the client.
5. In subsequent heartbeats, the leader notifies followers of the new **commitIndex**, instructing them to apply the data to their own state machines.

Safety

Raft ensures safety via the **Election Restriction**. Candidate for winner won't win election if its log is not in the most up to date state. A candidate will also not win an election if its **Term** is too old, this is, if it's bigger than 1, because nodes increase term by 1 every time a new leader is elected, so if Term difference is bigger than 2, then the node with lower term must have lost access to the cluster, because he skipped at least one leader.

During the voting process, a node will not vote for a candidate if the candidate's log is not at least as up-to-date as its own (comparing the term and index of the last log entry). This guarantees that committed data is never overwritten, preserving the State Machine Safety property.

2.5 Cache system architectures

Consensus can only take us as long as the coordination of data allows. For the system scalability to handle gigabytes of data and thousands of requests per second we need to use architectural designs.

2.5.1 Sharding and Consistent Hashing

Because we need a cache so big to use multiple servers, we need a way to split data between them. The process of splitting the data is called sharding [7].

Traditional Modulo Hashing

The most basic approach would just use $server = \text{hash}(key) \pmod{N}$, where N is the number of servers. This might be fine, if we had stable amount of servers and immediately could calculate how much data at most we will use. However if we don't know how much data we need, or if we don't want to pay for servers to use 5% of it's resources we would want to scale on the go.

With the basic approach of modulo, if a server is added or removed (N changes), the result of the modulo operation changes for nearly all keys. This will force us to move data between all of the server, forcing a massive rebalancing of data across the entire cluster. This is just inefficient and way too slow.

Consistent Hashing

Consistent hashing maps both data and servers onto a logical ring topology (0 to $2^{32} - 1$). A key is assigned to the first server encountered moving clockwise on the ring.

When a server is added or removed, only K/N keys need to be remapped (where K is the total keys), which significantly minimizes data movement.

But this has another problem. What if some part of the ring is used way more often then other? For this case we use **Virtual Nodes**. Each physical server is placed onto the ring multiple times (e.g., at 100 different positions). This statistical distribution ensures a more uniform load balancing across the cluster.

2.5.2 Role of the client: Thin Client vs. Smart Client

Of course the system is build for someone, a client. So there also needs to be made a decision on how the client will connect to the system. There are two broad options:

- **Thin Client (Proxy-based):** The client is unaware of the cluster topology. It sends requests to a load balancer or a random node (acting as a proxy), which then forwards the request to the correct shard.
 - *Pros:* Simple client implementation; topology changes are transparent to the application.
 - *Cons:* Additional network hop (Client $\rightarrow$ Proxy $\rightarrow$ Shard), increasing latency.
- **Smart Client:** The client maintains a local map of the cluster topology (slots mapping to node IP addresses). It computes the hash and connects directly to the target node.
 - *Pros:* Lowest possible latency (single hop); no bottleneck at a central proxy.

- *Cons:* Complex client logic; the client must listen for cluster configuration changes (e.g., failovers or resharding) to update its internal routing table.

2.6 Overview of existing solutions (Redis Cluster, Etcd) – Analysis in terms of consistency

To contextualize the solution developed in this thesis, it is valuable to analyze how established systems handle these trade-offs.

2.6.1 Redis Cluster

Redis Cluster is the industry standard for distributed caching. It employs a multi-master architecture with asynchronous replication.

- **Architecture:** It uses sharding with 16,384 hash slots. Each master node owns a subset of slots and has one or more replicas.
- **Consistency:** Redis prioritizes Availability and Performance (AP-leaning). By default, replication to slaves is asynchronous. If a master crashes after accepting a write but before replicating it to a slave, and that slave is promoted to master, the write is lost. Redis does not guarantee strong consistency; it provides eventual consistency and high throughput.

2.6.2 Etcd

Etcd is a distributed key-value store used primarily for shared configuration and service discovery (e.g., in Kubernetes).

- **Architecture:** It is built strictly on the Raft consensus algorithm. Unlike Redis, it does not shard data by default; every node replicates the full state machine.
- **Consistency:** Etcd is a CP system guaranteeing strict serializability. Every write goes through the Raft leader and must be committed to a quorum before returning success. This ensures data safety at the cost of lower write throughput compared to Redis.
- **Durability:** Etcd is a system that guarantees durability: Even if a node crash, the data is not lost.

The solution implemented in this thesis (Chapter 4) aligns more closely with the Etcd model regarding consistency (using Raft for strong guarantees) but incorporates

architectural elements of Redis (key-value storage focus) to explore the performance implications of strong consistency in a caching scenario. My solution **does not** guarantee durability, hence if the node crashes, the data needs to be refetch from durable data storage. This is one of the biggest differences between Etcd.

2.6.3 Analysis of hashing approaches

Karger’s Consistent Hashing

Proposed by Karger et al. at MIT in 1997, consistent hashing solves the reshuffling problem inherent in modular hashing. In a standard modulo setup ($\text{hash}(k) \pmod n$), changing n (adding/removing a node) results in nearly all keys being remapped. Karger’s approach maps both the **Nodes** and the **Keys** onto the same identifier space (a ring).

Why it was not used: While industrially popular (consistent hashing is used in Cassandra and Dynamo), it introduces complexity in the Controller’s metadata management. Maintaining a sorted ring and handling ”virtual nodes” requires more complex logic than a fixed partition map. For this thesis, I opted for a **Fixed Slot** approach (similar to Redis) because it allows for deterministic shard-to-node mappings that are easier to synchronize via Raft.

Range Partitioning (Sorted Monolithic Map)

This approach treats the entire key-space as a single, sorted monolithic map. The data is divided into contiguous ranges (e.g., keys $[A, K)$ on Node 1, $[K, Z)$ on Node 2).

- **Pros:** Allows for efficient range queries (e.g., `SCAN user:100:*`).
- **Cons:** Suffers from the “**Sequential Write**” problem. If keys are generated with a monotonically increasing prefix (like timestamps), all traffic targets a single node, creating a hot spot.

Why it was not used: A cache’s primary goal is often uniform load distribution for point Lookups (GET/PUT). Range partitioning is better suited for analytical databases (like HBase or CockroachDB). Given the use of Raft for the Control Plane, a hash-based distribution ensures even load across Data Nodes without the need for complex range-splitting logic.

FNV-1a Hashing Algorithm

The Fowler-Noll-Vo (FNV) hash 1a is a non-cryptographic hash function known for its extreme speed and ease of implementation. It works by multiplying the hash by

a large prime and XORing it with each byte of the input data.

- **Pros:** High throughput, low collision rates for short strings (ideal for cache keys), and minimal memory overhead.
- **Suitability:** Since this project requires mapping keys to a fixed set of shards during every client request, a low-latency hash function like FNV-1a is preferable over more complex cryptographic hashes like SHA-256 or MD5, which would introduce unnecessary CPU overhead.

2.6.4 Analysis of industry inspirations

Redis Cluster: The "Fixed Shards" Inspiration

Redis Cluster pre-shards the key-space into 16,384 "hash slots". A key is mapped to a slot using: $\text{slot} = \text{CRC16}(\text{key}) \pmod{16384}$.

- **Inspiration:** My implementation adopts this "Fixed Shard" model. By mapping keys to a constant number of shards (e.g., 10 or 1024), we decouple the data distribution from the number of physical nodes.
- **The Difference:** Redis uses asynchronous replication by default to maximize performance, which can lead to data loss. My system uses **Synchronous Replication** to ensure Strong Consistency, sacrificing a degree of write latency for the safety guarantees required by the thesis.

Amazon Dynamo: Availability vs. Consistency

Dynamo is the archetypal "AP" (Available, Partition-Tolerant) system. It uses consistent hashing and eventual consistency (Vector Clocks, Gossip protocol) to ensure it is "always on."

- **Inspiration:** Dynamo's concept of a "Preference List" for replication was an inspiration for how the Controller manages shard assignments.
- **The Difference:** Dynamo's philosophy is "eventual consistency." It allows conflicting writes and resolves them later. My project purposefully takes the opposite path: identifying as a "CP" system that uses Raft and fencing to prevent conflicts from ever occurring, matching the strict requirements of linearizability.

TiKV: Raft-per-Shard vs. Centralized Control

TiKV is a modern distributed KV store where every "Region" (shard) is its own independent Raft group.

- **Comparison:** TiKV's approach is the gold standard for massive scalability. However, running 1000 shards would mean running 1000 independent Raft state machines per node, which is computationally expensive.
- **My Approach:** To balance performance with safety, I separated the system into a **CP Control Plane** (Raft) and a **Synchronous Data Plane**. This significantly reduces the overhead; only the cluster metadata (topology) requires the full weight of the Raft algorithm, while data replication is handled by a simpler, faster synchronous Primary-Backup protocol.

Chapter 3

Requirements analysis and Software Architecture Design

3.1 Functional and non-functional requirements

We define functional requirements as what the system must do, specifying its behaviours and functions:

- Support GET, PUT, DELETE operations via HTTP api
- Automatic sharding of key-space across a configurable number of shards
- Automatic replication of each shard (primary + replicas)
- Dynamic cluster membership - data nodes can be added and removed at runtime
- Rebalancing when nodes join, leave or fail
- Smart client that routes requests to correct node and caches topology

We define non-functional requirements as how the system operates, what it guarantees in terms of performance, security, usability and readability:

- Strong Consistency - the system must reject requests when quorum cannot be reached, the system guarantees that all responses are correct
- Partition tolerance - system continues to operate even if some of the nodes fail or disconnect from others
- Fault tolerance - system detects node failures and triggers automatic fail-over
- No downtime when adding or removing new nodes and distributing shards between data nodes

3.2 High level architecture

The system is split into two logical parts. One of the part is Control Plane, the second one is Data Plane.

3.2.1 Separation of Control Plane and Data Plane

Control plane is responsible for:

- Runs a Raft consensus algorithm that stores cluster metadata - topology, shard mapping, node status, epoch counter.
- Exposes internal HTTP endpoints used by DataNodes for pull based shard migration
- Performs node failure detection (via the ‘Reaper’) and initiates rebalancing

Data Plane is responsible for:

- Stores the actual key-value data
- Handles client RPC (GET, PUT, DELETE)
- Reports heartbeats to the Control Plane

3.2.2 gRPC communication protocol and interface definition

gRPC is a high-performance Remote Procedure Call (RPC) framework developed by Google. It uses Protocol Buffers as its interface definition language and message serialization format, providing a binary-encoded alternative to traditional JSON-over-HTTP, which results in smaller payloads and faster processing.

The system utilizes a dedicated gRPC service for the Control Plane’s Raft operations. We define 6 core methods and 10 data types in the `raft.proto` file:

Table 3.1: gRPC Service Methods for Raft Consensus

Method	Responder	Purpose
Forward	Leader	Forwards a client write request to the Raft leader.
ForwardGet	Leader	Forwards a client read request to the Raft leader.
LogRequest	Follower	Standard Raft <i>AppendEntries</i> for log replication.
VoteRequest	All	Standard Raft <i>RequestVote</i> for leader election.
Heartbeat	All	Basic reachability check used in health monitoring.
InstallSnapshot	Follower	Transfers chunked state machine data to lagging nodes.

These functions form the backbone of the Control Plane's consistency. Detailed implementation of how these RPCs interact with the Raft state machine is discussed in Chapter 4.

3.3 Design of Control Plane module

3.3.1 Cluster topology and metadata management

Topology data contains:

- Nodes map: node ID mapped to NodeMetadata - address, status, last heartbeat timestamps
- Shards map: shard ID mapped to ShardMetadata - primary ID, replica ID, status
- Epoch - monotonically increasing identifier used for conflict-free updates

Controller updates cluster topology using Raft Algorithm. I want to make sure that Control Plane gives me a Strong Consistency, because if it wouldn't, then the system also wouldn't be Strongly Consistent. This is because, if there are conflicts in Control Plane, then the client might send request to wrong datanode, hence creating impossible state, which would not guarantee "correct response on every request".

Rebalancing is using Control Plane's active nodes to determine which node should be primary and which should be a replica. Then it writes new config to Raft Log, that is distributed between every other Controller in Control Plane.

3.3.2 Node failure detection mechanism

I implemented a Reaper mechanism which tracks last heartbeat timestamp of each nodes. In background, Reaper runs a thread that checks if node did heartbeat within configured grace period. If it didn't, then Reaper invokes a callback to Control Plane, triggering a topology rebalance.

3.4 Smart Client design

3.4.1 Query routing strategy and topology caching

The client on initialization connects to Control Plane to fetch a topology that it stores in local cache. On each request, the client hashes the key to shard ID and looks up the primary node from cached topology. Then the request is sent.

3.4.2 Handling retries and Failover mechanism

On RPC failure the client retries the request against current primary. If this primary is unreachable or the request is rejected, then the client will refetch the cluster topology.

Chapter 4

Implementation of the Raft Consensus Module (pkg/raft)

The core of the distributed system is the Raft consensus algorithm, located in the `pkg/raft` package. This module is responsible for managing the state of distributed nodes, ensuring data consistency across the cluster, and handling leader election and failure recovery. While the Data Plane uses synchronous replication for performance, the Control Plane requires the formal correctness and safety guarantees of Raft to manage the cluster's critical metadata.

The implementation is highly modular, decomposing the Raft protocol into several distinct components: `Elector`, `Replicator`, `DataSaver`, and `Snapshotter`. This separation of concerns allows for a clean concurrency model where each component operates on its own set of goroutines, communicating via channels to minimize lock contention.

4.1 Raft node state machine

The system defines the node's lifecycle through three primary roles: `Follower`, `Candidate`, and `Leader`. The transition between these roles is strictly controlled by the term logic and majority voting rules.

4.1.1 Implementation of roles: `Follower`, `Candidate`, `Leader`

The roles are defined as typed constants in `types.go`: `Follower`, `Candidate`, and `Leader`.

- **Follower:** The default state upon initialization (in `NewRaft`). Followers passively respond to RPCs from leaders and candidates.

- **Candidate:** A node transitions to this state when an election timeout occurs. This logic is encapsulated within the `startElection` method in `election.go`.
- **Leader:** The node that handles all client requests and coordinates log replication. Transition to this role is handled by the `becomeLeader` function in `utils.go`, which initializes the `Replicator` instances for all peers.

4.1.2 Concurrency and Locking Model

Unlike simple implementations that wrap the entire state in a single mutex, this module uses a `sync.RWMutex` for the core Raft state, while sub-components often use their own internal synchronization. A critical design choice was the delivery of committed logs to the `Application` interface; to prevent deadlocks and blocking the consensus loop, the mutex is explicitly unlocked during the `deliverToApplication` phase, allowing the application to process data without stalling incoming RPCs.

4.1.3 Leader election mechanism (`election.go`)

The `Elector` manages the node's election timer using a randomized timeout strategy (400ms – 800ms) to reduce the probability of split votes in large clusters.

1. **Timer Loop:** A dedicated goroutine runs `electionTimerLoop`, which listens on a `resetTimerCh`. This channel allows other components (like heartbeats or valid `AppendEntries` RPCs) to reset the timer efficiently.
2. **Safety Checks:** Before starting an election, the node verifies it is not currently the leader and not in the middle of installing a snapshot, as snapshot installation is a heavy I/O operation that might temporarily delay heartbeat processing.
3. **Parallel Requests:** When a node becomes a Candidate, it broadcasts `VoteRequest` RPCs to all peers in parallel using individual goroutines, ensuring the election process is as fast as the fastest majority of the network.

4.2 Log replication logic (`replicator.go`)

The replication logic is implemented using an isolated "per-peer" model. For every follower in the cluster, the leader spawns a dedicated `Replicator` goroutine.

4.2.1 Isolated Replicators and The Pulse Protocol

Each `Replicator` maintains its own state for a specific peer, including `nextIndex` and `matchIndex`. This provides several advantages:

- **Fault Isolation:** A slow or disconnected follower does not block replication to other healthy nodes.
- **Instant vs. Periodic Replication:** The replicator waits on a `signalCh`. When the leader receives a new client request, it "pulses" this channel to trigger immediate replication. If no writes occur, a 50ms ticker ensures periodic heartbeats are sent to maintain leadership.
- **Self-Healing Quorum:** If a replicator detects too many consecutive failures (defined in `maxConsecutiveFailures`), it triggers a `checkQuorumHealth()` call, which can cause a leader to step down if it realizes it no longer has a healthy majority.

Replication is triggered via the `LogRequest` RPC (standard Raft *AppendEntries*). The `Replicator` runs a continuous loop that monitors a signal channel. When triggered, it calculates the `prevLogIndex` and `prevLogTerm` to ensure log consistency. These arguments are packaged into a `LogRequestArgs` Protocol Buffer message and sent via the `PeerClient` interface.

4.2.2 gRPC Connection Management

Maintaining stable gRPC connections in a dynamic cluster is handled by the `ConnectionManager`.

- **Lazy Initialization:** Connections to peers are only established when needed (e.g., during the first replication attempt or vote request).
- **Exponential Backoff with Jitter:** If a connection fails, the manager uses a backoff strategy (starting at 100ms up to 5s) with added randomization (jitter). This prevents the "Thundering Herd" problem where all nodes try to reconnect to a recovering leader simultaneously, potentially overwhelming its network stack.
- **Health Monitoring:** The manager periodically verifies the state of idle connections, closing and reopening them if they become stale or if a underlying network partition is detected.

4.2.3 Committing entries (Commit Index) and applying to the state machine

The leader tracks the `matchIndex` (represented as `ackedLengths`) for each follower. When a majority of nodes acknowledge a log entry, the leader advances its `committedLength`. This logic is implemented in the `commitLogEntries` function in `raft.go`.

Upon commitment, messages are applied to the local state machine via the `Application` interface (defined in `application.go`). The application executes the command (e.g., updating the in-memory cache) and returns the result, which is then sent back to the client via a response channel.

4.2.4 The Log Request (`AppendEntries`)

The protocol uses a sophisticated `prepareLogRequestArgs` method to calculate the exact suffix of the log that needs to be sent. It handles the transition between sending incremental log entries and triggering a full snapshot installation if the follower's lag exceeds the available logs in memory.

4.3 Asynchronous Persistence Management

To survive crashes and restarts, the Raft module implements strict persistence guarantees using the `DataSaver` struct located in `persistence.go`.

4.3.1 Writing Raft state to disk (`persistence.go`)

The persistence layer uses Go's `encoding/gob` package to serialize data structures to disk. The `DataSaver` utilizes a worker pool pattern to handle save requests asynchronously, preventing disk I/O from blocking the main Raft consensus loop.

The state is split into two files:

- **Metadata File:** Stores the `currentTerm`, `votedFor`, and `committedLength`.
- **Logs File:** Stores the append-only sequence of `LogEntry` objects.

4.3.2 State Recovery

Upon restart, the `LoadValues` method performs a sequential scan of the log file, reconstructing the in-memory log slice. Because logs are appended, the system handles partial writes by ignoring corrupted `gOB` entries at the end of the file during recovery.

4.4 Log Compaction and Snapshotting

To prevent the log file from growing infinitely, the system implements a managed snapshotting mechanism.

4.4.1 Snapshot Trigger and Creation

When the log length exceeds the `snapshotThreshold` (configured in `Config`), the leader initiates a snapshot. It requests a serialized state from the `Application`, writes it to a `.snap` file, and then truncates the in-memory log.

4.4.2 Chunked Snapshot Transfer (`InstallSnapshot`)

Transferring a large snapshot over gRPC can be problematic as it might exceed the maximum message size or cause network timeouts. I implemented a `**Chunked Transfer Protocol**` in `sendInstallSnapshotRPC`:

- The snapshot file is read in 128 KB blocks.
- Each block is sent as a separate `InstallSnapshotRequest` containing an `offset` and a `Done` flag.
- The follower reconstructs the file using `WriteAt`, ensuring that the snapshot is durable on the follower's disk even if the transfer is interrupted and resumed.

This mechanism allows the system to synchronize new nodes even when the dataset is significantly larger than available network buffers.

4.5 System Configuration and Environment Parameters

The Raft module is highly configurable through environment variables, allowing it to be tuned for different network conditions or testing scenarios.

Table 4.1: System Configuration Parameters

Variable	Default	Description
<code>RAFT_ID</code>	<i>required</i>	Unique integer ID for the Raft node.
<code>TOTAL_NODES</code>	<i>required</i>	The total number of nodes in the cluster.
<code>RAFT_ADDRS</code>	<i>required</i>	Comma-separated list of all node addresses.
<code>SNAPSHOT_THRESHOLD</code>	<i>required</i>	Number of log entries before a snapshot is triggered.
<code>RAFT_INITIAL_BACKOFF</code>	1s	Initial delay for gRPC reconnection attempts.
<code>RAFT_MAX_BACKOFF</code>	30s	Maximum delay for reconnection attempts.
<code>RAFT_ARTIFICIAL_DELAY</code>	0ms	Simulates network/CPU lag for performance testing.

Chapter 5

Implementation of Control Plane and Data Plane

5.1 Implementation of Control Plane

Control Node is the central orchestrator of the entire application. It is responsible for maintaining the state of the application and ensuring that all nodes are in sync.

5.1.1 Topology state machine

The core of the Controller is a Replicated State Machine that is responsible for managing the cluster metadata. This ensures that in a multi-node setup all instances of Controllers are in sync about which Data Node holds which shard.

The State represents:

- Epoch - a monotonically increasing number representing the version of the topology. This works similarly to Raft term.
- Nodes - a map of all registered Data Nodes with information about their addresses and statuses.
- Shards - a map of all shard IDs to their Primary and Replica data node IDs.

The Controller implements the Raft application interface. It processes commands like node registration and topology updates. Changes to the topology are not applied until they are committed by the Raft algorithm to ensure strong consistency. When a command is sent to the Controller, it updates its local config and, if the Controller is the leader, it may trigger a rebalance. To prevent the Raft log from growing infinitely, the system supports snapshots, saving the current cluster configuration to durable storage.

5.1.2 Node health monitoring

A dedicated component called the Reaper is responsible for monitoring the system for failures. Data nodes are designed to periodically send heartbeats to the Control Plane, which updates the timestamp of the last received heartbeat for each node. The Reaper runs a background goroutine that wakes up every second to check the liveness of all data nodes. If a node has not sent a heartbeat within the configured grace period, it is marked as DEAD. In this case, the Reaper triggers a callback that signals the Controller to take action. If the failed node was a Primary, one of its replicas is promoted to the primary role. Subsequently, the rebalancing logic will attempt to find a new node to act as a replica to restore the desired redundancy level.

5.1.3 Shard rebalancing

Rebalancing is responsible to ensure that data availability and even distribution of data is preserved.

When a node failure is detected, the system performs an emergency promotion:

1. The controller identifies all shards where the failed node was the primary.
2. For each such shard, the first available replica is promoted to become the new primary. Since the original primary is dead, no data can be copied; the promoted replica relies on the data it already has.
3. The dead node is removed from the active node collection.

Note: When both primary node and replica dies, then the data in the shards is lost. This is a direct consequence of not providing a durable cache as noted previously. It is possible to increase number of replicas to two, but it will slow the cache, as we need to ensure that the primary data node sends data to every replica before it returns OK to the client. However, it's very unlikely to both nodes die at the same time if they are on different physical servers.

Rebalancing is triggered automatically when a new node registers or a failure occurs. It use a greedy strategy:

1. It identifies all shards that lack a primary node and assigns them to ACTIVE nodes using a round-robin approach.
2. It ensures that every shard has the required number of replicas, selecting new nodes to act as replicas if needed.

3. For initializing these new replicas or during planned load balancing, a synchronization protocol is used to ensure data consistency.

For initializing new replicas, the controller implements a 4-phase synchronization protocol:

1. The target node is instructed to perform a bulk copy of all data from the current primary node.
2. The shard is marked as LOCKED in the cluster topology. This temporarily stops writes to this specific shard to ensure a consistent state.
3. A final incremental catch-up is performed to transfer any data that was added during the initial bulk copy in phase 1.
4. The target data node is officially added to the replica set and the shard is set back to ACTIVE.

5.1.4 The NoOp Command

The NoOp command is a special administrative command in the Raft state machine that performs no data modification. Its primary purpose is to refresh the leader's knowledge of the current state and verify that it still holds a valid quorum. In this implementation, the Controller uses NoOp to clear any potential uncertainty after an election and to ensure that topology updates are only issued by a leader that is fully synchronized with the cluster.

5.1.5 Controller HTTP API Reference

The Control Plane exposes several internal and external HTTP endpoints. These are used by Data Nodes for registration and by the Smart Client for topology discovery.

Table 5.1: Controller HTTP API Endpoints

Endpoint	Method	Description
/register	POST	Registers a new Data Node with its address.
/heartbeat	POST	Receives health updates and returns current Epoch.
/topology	GET	Returns the full JSON description of the cluster state.
/pull	POST	Triggers a shard migration from source to target node.

While the emergency promotion handles sudden failures, the system also manages the lifecycle of nodes entering and leaving the cluster gracefully.

Scaling Up: The Lazy Assignment Strategy

When a new Data Node registers, the `Rebalance()` function is invoked. In the current implementation, this follows a **Lazy Assignment** strategy:

- New nodes are primarily used to fill "gaps" in the cluster— that is shards that do not have required number of replicas or shards that have no assigned primary data node.
- To prevent unnecessary data movement (which can saturate the network), the system does not automatically "steal" shards from existing healthy primary nodes.
- For active load balancing (manual redistribution), an administrator can trigger the `MoveShard` command, which utilizes the 4-phase synchronization protocol to migrate a primary role to the new node with zero data and minimum down-time.

Scaling Down and Replica Recovery

The system treats a deliberate node removal similarly to a failure. If a node is decommissioned:

- **Redundancy Restoration:** If the removed node was a replica, the `Rebalance` logic detects that the shard's replication factor has decreased. It automatically identifies another healthy node in the cluster and triggers a data pull to restore the backup.
- **Safe Promotion:** If the node was a primary, the promotion happens first, followed by the replica restoration on a different node.

This ensures that the cluster is **self-healing** regarding its durability guarantees; as long as there is spare capacity in the cluster, the system will always try to maintain the configured number of replicas for every shard.

5.2 Data Plane

5.2.1 Concurrent Data Store

The data storage provides a simple but thread-safe in-memory key-value store. It uses a standard Go map structure wrapped with a mutex. The data store supports data migration. In particular it supports two functions:

1. ExportShard(shardID, totalShards) - a crucial function for rebalacing, as it iterates through the entire map and hashes each key to determine its shard and returns only those keys that belongs to given shardID
2. Import(data) - it bulks load into the map. The function is used during shard migration or recovery.

5.2.2 Lease management and fencing

Lease management is the primary system defense against split-brain problem. The data nodes are implemented to send a heartbeat to the Control Plane to inform that they are alive. If the controller acknowledges the heartbeat then the data node extends its validity timestamp. Lease is a configurable value that says how long a data node is alive. We use fencing to check the lease by comparing current time to last validity timestamp. If the difference is bigger than lease time, then the data node immediately stop serving requests. To ensure that we always check this condition, every PUT, GET, DELETE operations check the lease. If the lease has expired, then the data node will return a 503 Service Unavailable HGTTP response, effectively fencing the node out of the cluster.

5.2.3 Synchronization and Consistency

Data Plane maintains consistency by coordination with the Control Plane.

Data Plane makes sure that the topology is always synchronized. The heartbeat responses contains the latest Epoch. If the Epoch is higher than the node's local Epoch value then lease manager will immediately fetch the latest topology from the Control Plane and update the local one. This ensures that the data node is always aware of its current role that the Control Plane assigned it to - whether it's a primary node or a replica.

Data Plane uses epoch based fencing. If a node has an active lease, every write request includes an Epoch. The data node compares the client's Epoch to its own. If the requests Epoch is smaller than the local one, then the request is rejected with 412 Precondition Failed, preventing stale writes from delayed clients that have to refetch the current topology.

The last synchronization mechanism is the Primary-Backup replication. If the data node is a primary node, then its responsibility is to replicate writes to all Replica nodes. This replication is **synchronous**. This causes the system to be Strong Consistent, because we only accept the request if all replicas has saved it and

the primary nodes had saved it as well. This also provides a form of durability, as the system will keep the data as long as at least one replica is alive.

Note: The strong consistency definition in Chapter 2.1.2 says that:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

So if all replicas received the write request and then the primary node died, we have uncertainty - the second case. But since all replicas said OK - then the client will see the request as accepted from now on. In the other case if some replicas received the write and the other did not and the primary node crashed, then we have a problem of some replicas having the data and others not. This is where the client solves this problem by client-side idempotency. This will be talked in more detail in client section.

5.2.4 Data transfer protocol

Shard synchronization is a crucial part of the system. There are two data nodes that need to synchronize their data. Let's call them source and target node. The source node is the one that exports its data, while the target node is the one that imports it. Data Plane uses a pull model where:

1. The controller sends a pull request to target node
2. The target node sends a GET request to source node export endpoint that returns the result of `ExportShard()` function (see Section 5.2.1).
3. The target node calls `Import()` function (see Section 5.2.1) with the data it received from source node.

5.2.5 Data Node HTTP API Reference

The Data Plane nodes expose endpoints for client CRUD operations and internal synchronization.

Shard Locking: During migration (Phase 2 of the 4-phase protocol), the shard is set to `StatusLocked`. Any client request to a locked shard is rejected with an HTTP 423 `Locked` status, signaling the Smart Client to wait and retry.

Table 5.2: Data Node HTTP API Endpoints

Endpoint	Method	Description
/data	GET	Retrieves a value by key if the node is the primary.
/data	POST/PUT	Stores a key-value pair and replicates to backups.
/data	DELETE	Removes a key and replicates the deletion.
/replicate	POST	Internal: Replica receives a data write from primary.
/delete-replicate	POST	Internal: Replica receives a delete command from primary.
/export	GET	Internal: Exports shard data for migration.
/import	POST	Internal: Imports shard data from a source node.

While the Control Node handles the cluster high-level orchestration and Data Node handles lower-level data logic, the Client and Access Library is responsible for efficient operations and ensuring reliable communication.

5.2.6 Smart client mechanism

The smart client is a implementation that eliminates the need for centrally implemented load balancer or proxy of the system that can increase latency or break the data consistency - talked in more details in Chapter 3.

The client maintains a local copy of the cluster topology that is fetched from the Control Plane. The client will query the one of the controllers at the start of the session to get the latest topology. The client has to hold its own Epoch to send in every request and also compare whether the state of topology is the latest. The client also holds the list of active data nodes and shard to node mapping to connect hash to the node.

Every GET, PUT, DELETE operation has a key associated with it. The client needs to calculate the hash of the key to send the request to correct node. The client will perform operations:

1. Hash the key using FNV-1a algorithm (see Section 2.6.3).
2. Determine shard id by taking the hash modulo number of shards.
3. Look up the primary data node id for that shard in local cache
4. Send the request to the primary data node id.

The client is lazy but reactive. It keeps using its local cache as long as its valid. The validity of a cache is determined by a datanode response:

1. Status 412 Precondition Failed - the Epoch of the client is too old, the data node has newer Epoch in its local values
2. Status 400 Not Primary - the shard has migrated or failover occurred

In both cases the client has to refetch the topology and all of the other data from Control Plane (described in Section 3.4.1) and update its local cache.

5.2.7 Adaptive Retry and Backoff Strategy

To ensure resilience against transient failures, the Smart Client implements an adaptive retry loop:

- **Retry Limit:** Each operation is attempted up to `maxRetries` (default: 5) times.
- **Role Miss Retry:** If a node returns `400 Not Primary`, the client immediately refetches the topology and retries the request on the new primary.
- **Migration Wait:** If a shard is locked (`423 Locked`), the client waits for 100ms before retrying, giving the migration protocol time to complete Phase 2.
- **Failover Wait:** If a node is fenced or unreachable (`503 Service Unavailable`), the client waits for 200ms to allow the Control Plane's Reaper to detect the failure and trigger an emergency promotion.

This tiered wait strategy balances system responsiveness with the need to avoid overwhelming a cluster that is currently undergoing a reconfiguration.

5.2.8 Client-side idempotency

To prevent duplicate operations during retries (especially when a primary node has died) the client uses idempotency token and client id fields in each request. This makes every single write and delete operation to use a unique UUID token that is used in every request. It is used to check the state of the operation. Each log in Raft algorithm is implemented to hold the idempotency token and client id. Because of this token if the client is unsure whether the operation succeeded, he can always resend the request - if the operations succeeded, nothing will happen, as the system will just find the previous request with the same token. If the operation failed, the system will retry it.

This ensures that in the Data Plane, the primary node acts as the coordinator for the token. When the writes are replicated to replicas, then the token is

also passed with them. If the primary node fails after replicating but ACK-ing the client, the promoted backup already have the request with token. Then if the client is unsure whether his request has failed or not, then the replica that has become a primary node will have the request with the same token and reply OK. This solves the previously discussed problem where only some replicas receive the write and others do not because the primary data node crashes (see Section 5.2.3).

Lets imagine a situation where we have a cluster with primary node P and two replicas R_1 and R_2 . The client sends a request `PUT(key, value)` with some token T . The primary node P replicates to R_1 , which saves the request. Then P crashes. The client request times out and the client is not in the state of uncertainty, so it will resend the request. We have two possibilities:

1. R_1 becomes the new Primary node - it will see that it has the request with the token and say OK. Because of how the Shard Synchronization works (see Section 5.2.4), then R_2 will also have this request in it's memory, so the state is correct.
2. R_2 becomes the new Primary node - this Replica didn't receive the request, so it will not have it in it's memory. So on the resend request it will write the request to it's memory and replicate it to R_1 which will overwrite the currently held request. The state is correct

It can be easily increased to any number of Replicas, as the argument is the same.

5.3 Control Plane API Layer (pkg/api)

The `pkg/api` package provides the external interface for the Control Plane. While internal cluster coordination uses gRPC, administrative tasks and Control Plane state queries are exposed via a traditional HTTP/JSON API.

5.3.1 Bridging Raft and HTTP

The `api.Server` acts as a wrapper around the Raft node. When an HTTP request arrives (e.g., to update a configuration), the server must ensure the request is handled by the current leader.

- **Leader Caching:** To avoid the latency of checking leadership status on every request, the API server maintains a `LeaderCache`. It caches the address of the known leader for a short period (1s), refreshing it only if a request to that leader fails.

- **Command Forwarding:** If a client hits a follower node with a write request, the follower uses the gRPC `Forward` method to send the command to the leader. The leader then proposes the command to the Raft log.

5.3.2 Idempotency and Resilience

Distributed consistency requires handling retries safely. The API layer implements two critical sub-components:

1. **Idempotency Cache:** To prevent the same administrative command from being executed twice (for example, registering the same data node multiple times due to a network timeout during the first attempt), the server tracks request IDs in an `IdempotencyCache`. It discards duplicate requests before they reach the Raft log.
2. **Auto-Retriener:** Transient failures are handled by a `Retriener` component. It automatically retries idempotent operations if the connection to the leader is interrupted during consensus, ensuring high availability of the Control Plane operations.

5.3.3 Control Plane Client API Reference

The following table lists the endpoints exposed by the API server for external interaction:

Table 5.3: Control Plane Client HTTP API

Endpoint	Method	Description
<code>/kv</code>	POST	Proposes a generic KV update to the Raft state machine.
<code>/kv/{key}</code>	GET	Reads a value from the replicated state machine.
<code>/kv/{key}</code>	DELETE	Deletes a value from the state machine.
<code>/health</code>	GET	Returns the liveness status of the controller node.
<code>/status</code>	GET	Returns technical Raft details (Term, CommitIndex, Role).
<code>/leader</code>	GET	Returns the address of the current cluster leader.

Chapter 6

Correctness verification and Fault injection testing

6.1 Methodology for testing distributed systems

As testing distributed systems requires independent servers I considered two options. First option was to run tests using multiple computers. This option however wasn't developer friendly. Every time I would want to test my system I had to boot multiple computers, synchronize data through version control system, then set it up. This would take too much time and effort just to run some basic testing. That's where the second option came in - I simulated distributed servers using Docker Containers. As I can control almost everything about Docker Containers - restarts, failures, networks, this was my way of testing.

6.1.1 Docker-based Simulation Environment

To facilitate consistent and repeatable testing, the entire system is orchestrated using `docker-compose`. The current setup simulates a cluster composed of 3 Controllers and 5 Data Nodes. This configuration is defined in the `docker-compose.yml` file, which manages:

- **Network Isolation:** It defines two distinct bridge networks: `raft-cluster` for internal gRPC communication between Raft nodes, and `client-access` for external API requests.
- **Persistence:** Volumes are mapped for each controller to ensure that Raft logs and snapshots survive container restarts, allowing for testing of recovery mechanisms.
- **Service Dependencies:** Data nodes are configured to depend on controllers, mirroring the real-world startup sequence where nodes must register with a

functional control plane.

This containerized approach allows for complex failure scenarios (like killing a node or partitioning the network via `docker network disconnect`) to be automated in scripts.

6.2 Unit and integration tests for the Raft implementation

As noted in chapter 3, Raft is a central thing in Control Plane, and Control Plane is base for everything in the system. Because of that my Raft algorithm implementation had to be throughoutly tested. I implemented unit tests for every major method in the system with edge cases. In particular I have unit tests of:

- elections - basic check whether it works on 3 and 5 nodes, then tests around replying to various Term value, then Election timeout test
- messages - creation, conversion between messages struct and grpc definitions
- persistence - saving logs, loading logs, comparing whether loaded logs are correct, saving empty logs, saving to invalid paths, saving to corrupted file
- snapshot - whether indexes in log are correct after snapshot, whether snapshot is saved correctly - after load to we have correct data, checking whether logs is truncated after snapshot, test for snapshot installation rpc, checking whether snapshot will run if below threshold or log empty, checking whether snapshot creates with any directory problems

But as far as unit tests can go, they cannot test how parts of the system behaves in terms of each other. There come integration tests:

- Full test of election, writing to a log and replication of this log to other nodes
- Test of forwarding to a leader if broadcast is sent to non-leader node
- Test of Raft recovery - whether node will have correct values after it crashes

There was also a benchmark created for Raft only. It tried to run as many requests per second as possible. On Macbook Air M1 2020, 16GB RAM it achieved around 290k requests per second.

6.3 Simulation of failures and network partitioning

While the Raft tests are useful for testing Raft, they are not useful for testing the entire system. System requirement was to be Strongly Consistent and have Partition

Tolerance. To verify these properties, I developed a suite of automated shell scripts located in `tests/fault_injection/` and `tests/e2e/`.

6.3.1 Fault Injection and End-to-End Test Suite

The two primary verification scripts are:

1. `test_network_partition.sh`: This script verifies Partition Tolerance by executing the following sequence:
 - Start the system and wait for it to reach a stable state.
 - Identifies the current Raft leader and disconnects it from the network.
 - Waits for the remaining nodes to detect the failure and elect a new leader.
 - Modifies the cluster topology (e.g., adding a node) via the new leader.
 - Reconnects the original leader and verifies that it successfully synchronizes with the new majority and reflects the changes.
2. `test_strong_consistency.sh`: This script verifies the system's linearizability by performing multiple sub-tests:
 - First configured some topology of the system to look like it was running for some time to check read-after-write on it. The test run PUT request for some random value to the key and without waiting asked for the value. The expected value of read was the write value.
 - Second was to write sequence of values to the same key (change the key value multiple times). Then there was one singular read sent. The expected value was the value of the last write.
 - Third part was similar to first, but this time there were multiple keys used.
 - Fourth part was about Control Plane consistency, the test tried to register a new node and expected the registration to succeed and be visible in following read
 - Fifth part was to expect all Raft nodes to be available after all of these operations from previous subtests
 - Sixth part was to write a value to some key and restart data-node holding it. The expected outcome was to data-node to have this value in the memory.
 - Seventh part was a rerun of second part, to make sure that data-node after restart is still functioning in the intended way.
 - Eight, final part was to write a value to data-node and then immediately do a read-after-write check whether replica has this data.

3. **basic_e2e_test.sh**: Located in the `tests/e2e/` directory, this script performs a full lifecycle test: starting the cluster, registering nodes, storing a large set of keys, and verifying they can be retrieved after a series of controlled node restarts.

All of these tests succeeded, guaranteeing that the system is Strongly Consistent and Partition Tolerant.

6.4 Continuous Integration (CI/CD)

To maintain code quality and prevent regressions during development, I implemented a Continuous Integration pipeline using **GitHub Actions**. The workflow is defined in `.github/workflows/go.yml` and performs the following steps on every pull request or push to the main branch:

- **Environment Provisioning**: It sets up a Linux environment with the required Go version (currently 1.24).
- **Dependency Caching**: To optimize execution time, the pipeline caches Go modules and build artifacts, typically reducing the build stage duration by over 50%.
- **Automated Testing**: It executes the command `go test ./... --short`. The `--short` flag is used in the CI environment to skip long-running benchmarks or extremely heavy integration tests while still verifying the correctness of all core logic, including Raft state transitions and data node CRUD operations.

This automated feedback loop was essential for the rapid development of complex consensus logic, where a small change in term handling could otherwise have catastrophic effects on cluster stability.

Chapter 7

Performance analysis and Comparison with Existing Solutions

7.1 Benchmarks

The main idea about a cache is its speed. If we did not care about speed, then using database would be better. In that case, the performance analysis is crucial.

I decided to run benchmarks to calculate how many requests per second the cache can support. Benchmark works for 10 seconds, using 100 workers. It writes as many request as it can to different key each request. After ten seconds, we divide the number of total requests by 10 to get the score of requests per second. //

For benchmarks I was using Macbook Air M1, 2020 with 16GB of RAM on MacOS Tahoe 26.1.

7.1.1 Benchmark Toolset

To ensure objective and reproducible results, I developed a suite of benchmarking tools located in the `tests/benchmark/` directory:

- `benchmark_rps.go`: A high-performance Go-based benchmark that measures maximum RPS using goroutines and connection pooling.
- `benchmark_scaling.sh`: Orchestrates tests across multiple cluster configurations to measure horizontal scaling efficiency.
- `benchmark_raft_delay.sh`: Evaluates the impact of consensus latency on Control Plane performance by injecting artificial Raft processing delays.

- `benchmark_comparison.sh`: Automates the deployment of both this system and competitor systems (Etcd, Redis) to provide direct side-by-side performance comparisons.

7.2 Throughput study (RPS) depending on the number of clients and shards

Table 7.1: Client Scaling (1 Datanode)

Workers	RPS
1	5,714
10	18,855
50	28,336
100	29,605
200	27,661

Table 7.2: Horizontal Scaling (Multiple Datanodes, No Replication)

Datanodes	Combined RPS	Scaling
1	28,460	1.0x
2	55,001	1.93x
3	72,731	2.56x

Table 7.3: Replication Impact (per-node throughput)

Configuration	RPS
1 datanode (no replication)	28,909
2 datanodes (no replication)	27,100
3 datanodes (no replication)	27,468
2 datanodes (WITH replication)	26,108
3 datanodes (WITH replication)	27,657

7.3 Analysis of Raft protocol overhead on response time (Latency)

I created a benchmark for a case where we used Raft for every single operation. In reality we do not do this, because our smart client hashes the topology and doesn't need to use Raft on every request.

However in the benchmark I made the client send every request through Raft. This resulted in around 8K requests per second. Then I implemented an artificial sleep for Broadcast and LogRequest RPC operation. It accepts how many milliseconds it

should sleep for before doing the part. This can showcase how important is Raft for the application. After running the benchmark scores were:

Table 7.4: Analysis of Raft protocol overhead on response time

Artificial Delay	Consensus RPS	Impact
0ms	8002	Baseline
10ms	274	29x slower
25ms	100	80x slower
50ms	59	135x slower

This shows how big of a difference can Raft speed make for the system.

7.4 Comparative study

To verify whether the score is good, we should compare it against something. I identified two open-source system best to compare against. First one is Etcd - also a Raft based implementation of key-value store that guarantees strong consistency. However Etcd is a durable key-value store, meaning that even if nodes do crash, the data is not lost.

The second system is Redis, as it's the most used cache in the world [9]. Redis however, is an AP (Availability + Partition Tolerance) system, meaning that it doesn't guarantee strong consistency. It is eventual consistency system, made to give the best possible performance. Redis is an 15 year old, C written, highly-optimised cache. It is very hard to write something better than Redis, so it's a good comparison in terms of speed.

7.4.1 Performance comparison with Etcd

Etcd was ran in Docker. I used official package *go.etcd.io/etcd/client/v3@v3.5.11* for client.

I ran 10x 10 second, 100 workers setup described in chapter 7.1. I saved amount of requests per second for every single of these runs, then averaged it. The score was 26661 Requests Per Second.

7.4.2 Performance comparison with Redis

Redis was also ran in Docker. The client used was official *github.com/redis/go-redis/v9*. Just like in Etcd, 10x 10 second, 100 workers benchmarks were run. The score was 38776 Requests per second.

7.4.3 Conclusions from the comparative analysis

I re-run my system 10x 10 second, 100 workers benchmarks alongside Etcd and Redis. It averaged 27715 Request per second, making it 3.95% faster than Etcd and 30% slower than Redis.

This score is expected. My system should be faster than Etcd and slower than Redis. Based on analysis of Raft protocol overhead on response time, the better and more optimised my Raft implementation would be, the faster it should work. My system could be working faster, but it would never reach speed of Redis because of its low level implementation, lenient consistency guarantee and higher level of optimizations.

Chapter 8

Summary

8.1 Evaluation of the degree of achievement of the work's objectives

My system functional and non-functional requirements have all been met. For each and every one of them there was a test written.

8.1.1 Encountered implementation problems and challenges

The biggest problem of the project was the implementation. As the original paper didn't have any implementation details, only a broad look of the algorithm. All operation implementation and handling concurrency was done by me.

Firstly I implemented Raft with multiple mutexes. Using multiple mutexes made it harder to reason about. That caused many deadlocks that I had to debug.

My second problem with Raft implementation was that I did not start with snapshot logic instantly. This made me rewrite almost all of the logic to adjust log indexes and terms to include snapshotted terms.

Then, even after implementation of snapshots there were not efficient. My initial strategy was to send snapshot during one request. This made the request block the thread and basically hold the mutex for longer, which made the server (leader) sending snapshot to a follower unresponsive. This created a bug, where the third server, that was not receiving snapshot saw a leader as a dead node and started the election. Because of correct values and logic, the server that was receiving snapshot accepted the new leader Vote Request and made the third server a new leader. My solution to this, was to split snapshot into smaller blocks. This removed unresponsibility of leader and made cluster work correctly.

8.2 Directions for further development of the project

As noted in chapter 7, the Raft speed determines entire application speed. I've found multiple tips and indentified possible directions for better performance of said algorithm. These include:

- **Batching requests.** Currently logs are saved every single request. This can be sped up, by batching them - sendings two or three log entires in one requests, instead of three of them. Saving logs opens and closes a file, which is a slow I/O operation. If the system would open only 33% of the time, it would greatly speed up the entire algorithm. This is however hard to implement, as if we accept a request and return OK response, then to achieve strong consistency we need to have the data in stable storage. If the node crashes before saving it (while waiting for 2/3 log entires for batch) it will completely break strong consistency.
- **Saving logs to file is a slow I/O operation that reopens file multiple times.** The idea is to use some heuristic to keep the file open while some variables occur. One possible variable is that, there might be way more requests and logs to save. If we did not have to close and open the file, it would make it work almost as fast as batching, but without the risk of breaking strong consistency. This is also not so easy to implement. We need to make sure that the `fsync()` calls are often, we also need a very good heuristic for when to close the file. If any bug occurs to the heuristic, then entire algorithm stops working as it's basing everything on log persistance.
- **Based on profiler, the encoding/decoding binary writing is taking around 40% of the entire time of application.** There might be introduced custom encoding and decoding that might be more specific and save everything needed in specific bits of integer, hence making the entire encoding shorter.
- **The I/O operations are long, the system might benefit from using the fastest threads to handle these operations.**

Bibliography

- [1] E. Brewer, "CAP twelve years later: How the "rules" have changed," in *Computer*, vol. 45, no. 2, pp. 23-29, Feb. 2012.
- [2] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, 2014, pp. 305-319.
- [3] T. J. Edwards et al., "CockroachDB: The Resilient Geo-Distributed SQL Database," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1493-1509.
- [4] MongoDB, "Replica Set Protocol Versions," [Online]. Available: <https://www.mongodb.com/docs/manual/core/replica-set-protocol-versions/>. [Accessed: 20-Dec-2024].
- [5] Neo4j, "Causal Clustering Architecture," [Online]. Available: <https://neo4j.com/docs/operations-manual/current/clustering/>. [Accessed: 20-Dec-2024].
- [6] RabbitMQ, "Quorum Queues," [Online]. Available: <https://www.rabbitmq.com/docs/quorum-queues>. [Accessed: 20-Dec-2024].
- [7] Ultima Online, "The Origin of Sharding," [Online]. Available: https://en.wikipedia.org/wiki/Database_sharding. [Accessed: 20-Dec-2024].
- [8] DB-Engines, "Ranking of Key-Value Stores," [Online]. Available: <https://db-engines.com/en/ranking/key-value+store>. [Accessed: 20-Dec-2024].